FreeRTOS

Author: Brian Kang
Create: 3/21/2026
Last Modified: 4/24/2026

Preface

This application note is for FreeRTOS, and based “Mastering-the-FreeRTOS-Real-Time-Kernel.pdf”
For more detail, visit http://www.freertos.org

RTOS is time critical task scheduler. It is a correction of C libraries.

Below are fundamental items you need to know.

Item	Description
OS	Infinity program that service the application program
multi-processor	Multi CPU
Process	Unit of program that is executed on OS
multi-process	Multi program
thread	Light-weight process. That has a separate stack and registers
Real Time	Execute program within defined time.
RTOS	Real Time OS that base the task.
task	Light-weight thread without Virtual Memory.
RTOS port	Compiler and Architecture Environment for RTOS Support

What is different between RTOS vs General Embedded Linux?

Feature	FreeRTOS	Linux (GPOS)
Size	Kilo Bytes	Mega Bytes / Giga Bytes
Real-Time	Deterministic (Hard)	Generally Non-Deterministic (Soft)
MMU	Not required	Requires Memory Management Unit
Boot Time	Near-instant (< 1ms)	Seconds to Minutes
Complexity	Low (Single address space)	High (User vs. Kernel space)
Board	MCU	SoC

RTOS core logic

Core Logic	Description
Scheduling	Decide what task will be executed at particular time. - task Time tick - Task Priority - task state - preemptive and round-robin
ITC	Inter-Task communication. Queue: - FIFO - Fixed size. - Queue by copy.

		- check empty on Receiver, check full on Sender
	semaphore	- binary semaphore and count semaphore - For synchronization.
	mutex	- For mutual exclusion. - Token concept – with priority inherit. To solve the priority inversion.
	Event Group	for a combination of one of more events to occur.
	Task Notification	Faster then other communication methods
	.	
ISR	Interrupt Service Routine - Deferral Execution. - Separated APIs (Interrupt Safe API)	

Popular RTOS

	Description	License	Owner
FreeRTOS	Small IoT Sensor, Battery Control	Open Source. MIT	Amazon
VxWorks	Aerospace, Industrial Robotics	Commercial	Wind River
Zephyr	Modern Wearable / BLE Device	Apache 2.0	Linux Foundation
QNX	Automotive ADAS / Cockpit / Medical Devices	Free on non-commercial	Black Berry

QNX and FreeRTOS are both real-time operating systems (RTOS) designed for different complexity levels. QNX is a high-performance, commercial microkernel OS (used in automotive/medical) for microprocessors, whereas FreeRTOS is a small, open-source RTOS kernel for microcontrollers. QNX offers robust safety, security, and complex features (graphics, networking).

	QNX Neutrino RTOS Key Features	FreeRTOS Key Features:
Architecture	Microkernel, where components run in user space to prevent total system failure.	Extremely small footprint, task-oriented kernel.
Use Cases	Automotive infotainment, medical devices, advanced robotics, safety-critical systems.	Resource-constrained IoT devices, small sensors, motor control.
Target Hardware	High-performance microprocessors (ARM Cortex-A, x86).	Microcontrollers (ARM Cortex-M).
Cost/Support	Commercial licensing fee, high-end dedicated support, certified safety features.	Open-source (free), widely adopted, supported by Amazon IoT integration.
Features	POSIX-compliant, advanced networking, advanced graphics stacks.	Simple, highly portable, limited built-in features (designed to be thin).
Complexity	QNX is a "full" operating system	FreeRTOS is a simple kernel
Memory	requires significantly more RAM/Flash	runs on very limited hardware
Fault Tolerance	allows restarting failed components without stopping the system.	
Licensing	QNX is paid/commercial	free

Ultimately, you would use QNX for a secure, complex system like a car's digital dashboard and FreeRTOS for a simple, low-cost sensor node.

Compare ARM Cotex-M and Cotex-A

	Cortex-A	Cortex-M
Performance & Complexity	Cortex-A excels at high-throughput processing, often with multi-core configurations and gigahertz speeds.	Cortex-M is optimized for energy efficiency and lower cost, typically operating in the low MHz range.
OS	Cortex-A runs high-level operating systems (HLOS) like Linux, Android, or Windows, enabled by their Memory	Cortex-M usually runs Real-Time Operating Systems (RTOS) or "bare metal" code (no OS).

	Management Unit (MMU).	
Memory Management	Cortex-A uses an MMU, allowing it to manage virtual memory.	Cortex-M uses a Memory Protection Unit (MPU) for memory protection but cannot handle virtual memory.
Architecture	Cortex-A often utilizes 64-bit support (ARMv8-A).	Cortex-M is strictly 32-bit (v6-M, v7-M, v8-M).
Use Cases	Smartphones, tablets, luxury handsets, automotive infotainment, gateways.	IoT sensors, wearables, industrial actuators, medical devices, Arduino.
	ARM Cortex-A processors are designed for high-performance, complex computing applications (smartphones, Linux systems) requiring virtual memory and high speeds.	Cortex-M processors are optimized for low-power, deterministic real-time tasks (IoT, microcontrollers) requiring low power consumption and simple OS support, such as RTOS.

1. Prepare FreeRTOS

Visit http://freertos.org, and download. It will be below directories and files.
The combination of compiler and processor is called as "**FreeRTOS port**".

```
.\
+- FreeRTOS\
|  +- Demo\
|  +- Source\
|  |  +- examples\
|  |  +- include\
|  |  +- portable\
|  |  |  +- ..
|  |  |  +- GCC\ [compiler]
|  |  |  |  +- RISC-V  [architecture]
|  |  |  |  +- RX600
|  |  |  |  +- MemMang [example heap allocation schemes]
|  |  |  |  +- ..
|  |  |  +- Renesas/
|  |  |  |  +- RX600
|  |  |  +- ..
|  |  +- tasks.c       [common source files on All ports ]
|  |  +- croutine.c
|  |  +- event_groups.c
|  |  +- list.c
|  |  +- queue.c
|  |  +- stream_buffer.c
|  |  +- timer.c
|  +- Test\
|  +- ..
+- FreeRTOS-Plus\      [for other FreeRTOS and ecosystem libraries]
|  +- Demo\
|  +- Source\
|  +- Test\
|  +- ThridParty\
|  +- VisualStudio_StaticProjects\
+- tools\
```

If you are using RISC-V with GCC, your FreeRTOS port is "FreeRTOS/Source/portable/GCC/RISC-V.

FreeRTOS has common file for All Ports on “FreeRTOS/Source/portable”, for example task.c, timer.c.
The FreeRTOS-Plus is for the libraries.

1.1 Configuration

Include Paths

1	The path to the core FreeRTOS kernel header files, FreeRTOS/Source/include.
2	The path to the source files specific to the FreeRTOS port in use, FreeRTOS/Source/portable/[*compiler*]/[*architecture*].
3	The path to the correct FreeRTOSConfig.h header file.

FreeRTOS is configured by “FreeRTOSConfig.h” on your project directory.
If your based demo project is “FreeRTOS\Demo\RX600_RX630-RSK_Renesas\RTOSDemo”, use FreeRTOSConfig.h file within this directory.
On your source code, it will use **#include “FreeRTOS.h”** to include the FreeRTOSConfig.h.

1.2 Create a FreeRTOS Project

Each FreeRTOS port comes with at least one demo application. Base the pre-configured demo project.
(Copy the demo project, and then remove unnecessary codes). For example:

main()

```
#include <FreeRTOS.h>

int main(void)
{
 prvSetupHardware(); /* setup your hardware */

 /* --- Application TASKS can be here */
 xTaskCreate(vTaskFunc, “Task 1”, 1000, NULL, 1, NULL);

 vTaskStartScheduler();

 for( ;; ) ;
 return 0;
}
```

vTaskFunc

```
void vTaskFunc(void *param)
{
    int local;
    static int my_static;

    for(;;) {

    }
}
```

1.3 Data Types and Coding Style

<portmacro.h>

TickType_t	For tick interrupt. Hold the tick count value. configTICK_TYPE_WIDTH_IN_BITS 16,32 or 64. Architecture dependent.
BaseType_t	Base on the architecture. Used for the return type. ex:pdTRUE/pdFALSE

Variable name

Prefix	Description
c	char
s	int16_t (short)

l	int32_t (long)
x	for BaseType_t and other non-standard types (structure, task handle, que handle,..)
u	for unsigned. ex: uc, us, ul, ux
p	pointer
e	enumerate
v	void

Function name

Return type and the file that is defined.

	return	file
v**Task**PrioritySet()	void	tasks.c
x**Queue**Receive()	BaseType_t	queue.c
pv**Timer**GetTimeID()	Pointer to void	timers.c

Macro Name

Lower case for defined location + Upper case

Prefix	example	defined location
port	**port**MAX_DELAY	portable.h or portmacro.h
task	**task**ENTER_CRITICAL()	task.h
pd	**pd**TRUE	projdefs.h
config	**config**USE_PREEMPTION	FreeRTOSConfig.h
err	**err**QUEUE_FULL	projdefs.h

Macro	value
pdTRUE or pdPASS	1
pdFALSE of pdFAIL	0

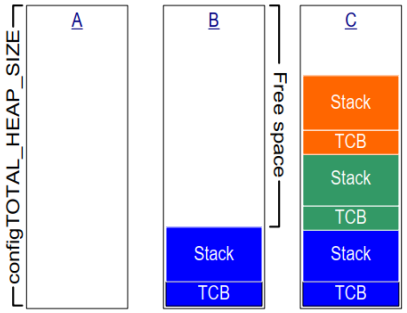
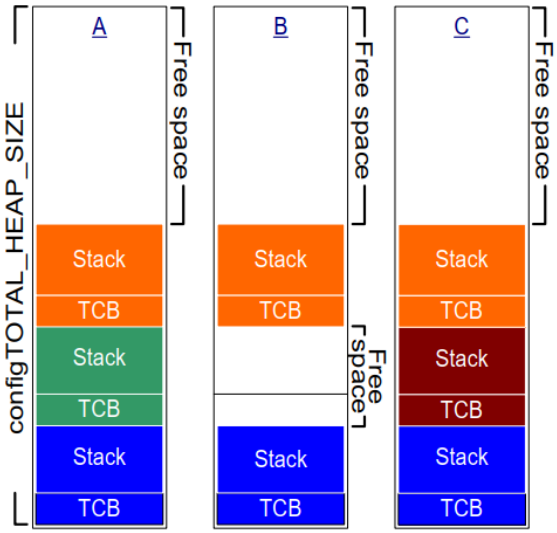
FreeRTOS-Plus

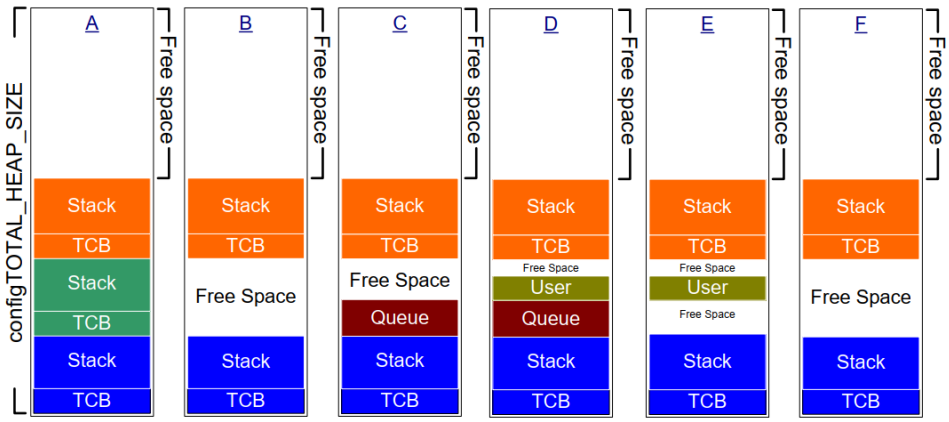
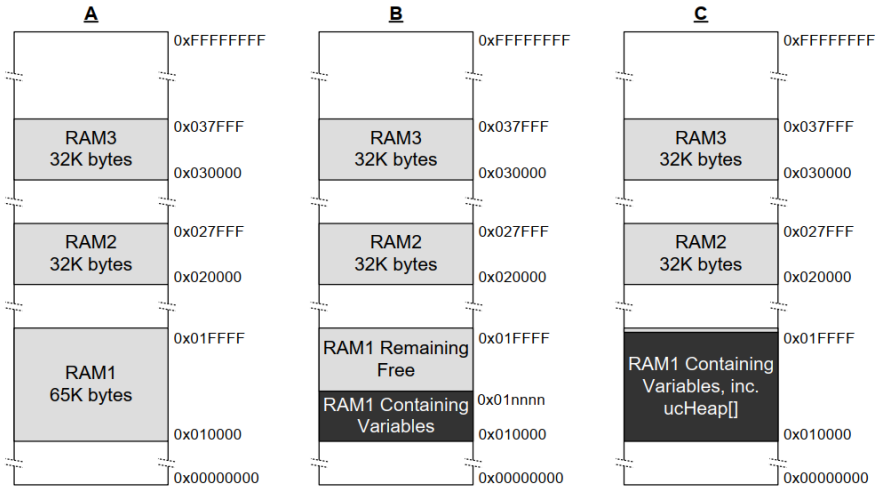
Module	Description
TCP	socket based, lightweight TCP/IP stack
CLI	command line input
IO	open(), read(), write(), ioctl()
Trace	debug

TAB is 4x space characters

2 Heap Memory Management

The kernel objects are **task**, queue, semaphore and event groups, and these objects can be allocated with static (at compile time) or dynamically (on run-time).
On dynamic allocation, it using pvPortMalloc() and vPortFree() function, instead of malloc() and free().
FreeRTOS provides 5 dynamic memory allocation schemes.

scheme	Description
heap_1	For small dedicated embedded systems, to only create tasks (kernel objects) before starting scheduler. **No vPortFree().** memory remains allocated for the application lifetime. For commercially critical and safety-critical systems. configTOTAL_HEAP_SIZE. Two pvPortMalloc(), one for TCB(Task control block), and the other for the task stack. 
heap_2	superseded by heap_4. **Best-fit algorithm**, it does implement vPortFree(). 
heap_3	use malloc() and free(). configTOTAL_HEAP_SIZE is not used.
heap_4	by subdividing an array into smaller blocks. use a **first-fit algorithm**. it combines adjacent free block into a single larger block.

<table>
<tr><td></td><td> 
</td></tr>
<tr><td>heap_5</td><td>heap_5 can combine memory from multiple separated memory spaces into a single heap. It uses,
void vPortDefineHeapRegions(const HeapRegion_t * const pxHeapRegions);
 
</td></tr>
</table>

Example: declare “.my_heap” section, and place at 0x2000000 by IAR compiler.

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] __attribute__ ( ( section( ".my_heap" ) ));
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] @ 0x20000000;
```

3 Task Management

The task needs to be implemented with below prototype that has a void pointer parameter and returns void.

```
void vATaskFunction(void * pvParameters);
```

For example:

```
void vTaskFunction(void *parameters)
{
 long lStackVariable = 1;
 static long lStaticVariable = 0;

 for( ;; )
 {
     //..code to implement
 }

 vTaskDelete(NULL);
}
```

3.1 Task Create

Each task requires two blocks of RAM: one to hold its Task Control Block (**TCB**) and one to store its **stack**.
Task can be below types.

type	Description
static	With static, use pre-allocated blocks of RAM. without static, allocate ram dynamically
restricted (or unprivileged)	With restricted, create tasks that execute with **limited** access to the system's memory. Without restricted, create tasks that execute in "**privileged** mode" and have access to the system's entire memory map.
Affinity	On SMP (Symmetric Multi Processor), specify which core for task.

Below is a basic xTaskCreate() function.

	Description
`BaseType_t xTaskCreate(`	or xTaskCreateStatic()
`  TaskFunction_t pvTaskCode,`	pointer to Task Code
`  const char * const pcName,`	Task name for human-readablity. configMAX_TASK_NAME_LEN
`  configSTACK_DEPTH_TYPE usStackDepth,`	sizeof stack or xTaskCreateStatic() for static size. for 32-bit wide, 128 (512byte = 128*4). configSTACK_DEPTH_TYPE
`  void * pvParameters,`	Parameter for task.
`  uBaseType_t uxPriority,`	priority 0 ~ (configMAX_PRIORITIES-1)
`  TaskHandle_t * pxCreatedTask`	to store a handle
`);`	// return values pdPASS or pdFAIL

And below example create two same priority tasks.

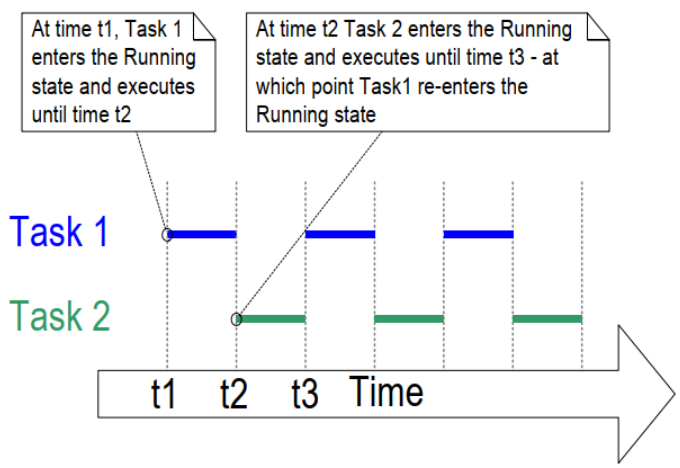
```
void vTask1( void *pvParam)
{
  volatile unsigned long ulCount;
```

```
void vTask2( void *param )
{
  volatile unsigned long ulCount;
```

```
  for( ;; ) {
    vPrintLine("Task1 is running" );
    // delay for period.
    for(ulcount = 0;
      ulCount < mainDELAY_LOOP_COUNT;
      ulCount++)
    {
      //...
    }
  }
}
```

```
  for( ;; ) {
    vPrintLine("Task2 is running" );
    // delay for period.
    for(ulcount = 0;
      ulCount < mainDELAY_LOOP_COUNT;
      ulCount++)
    {
      //...
    }
  }
}
```

```
int main(void)
{
 xTaskCreate(vTask1, "Task 1", 1000, NULL, 1, NULL);
 xTaskCreate(vTask2, "Task 2", 1000, NULL, 1, NULL);

 vTaskStartScheduler();

 for( ;; );
}
```



Both tasks execute on the same processor core, in reality, both tasks are rapidly entering and exiting the Running state. Both tasks are running at the same priority and so share time on the same processor core.

3.2 Task Priorities

uxPriority parameter on vTeskCreate() is used to assign the initial priority.
vTaskPrioritySet() can change priority.
configMAX_PRIORITIES has max value, normally 32. (64 on LP64)

	Description	Assigned as
Hard Real Time	a missed deadline is considered a **total system failure**. There is zero tolerance for delay.	High Priority
Soft Real Time	deadlines are important, but missing one is **not catastrophic**. The value of the result diminishes after the deadline, but it is still useful.	Low Priority

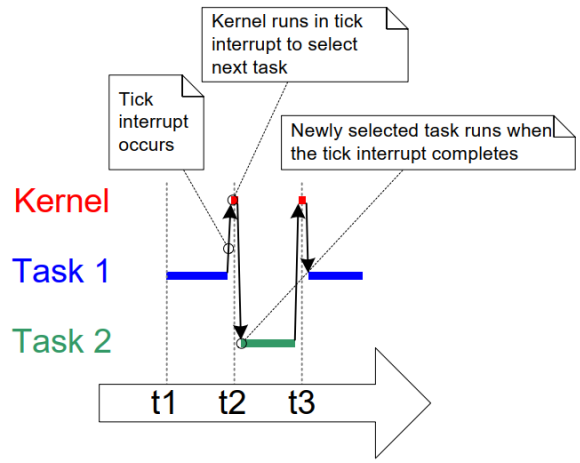
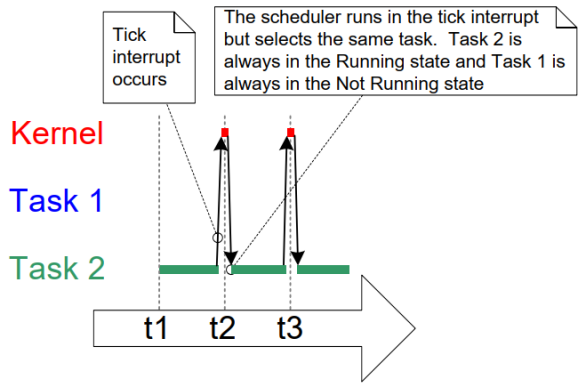
3.3 Time slice and Tick Interrupt

If tasks have same priority, each task executed for a **'time slice'**, entering the *Running* state at the start of a time slice, and exiting the *Running* state at the end of a time slice.
The scheduler executes at the end of each time slice to select the next task to run.
A periodic interrupt, called the **'tick interrupt'**, is used for this purpose. The time between two tick interrupts is called the **'tick period'**—so one time slice equals one tick period.

The scheduler will also select a new task to run immediately after the currently executing **task enters the *Blocked* state**, or when an **interrupt moves a higher priority task into the *Ready* state**.

```
configTICK_RATE_HZ (typical:100, 100Hz, 10ms),  pdMS_TO_TICKS()
```

Below show execution behavior to compare priority

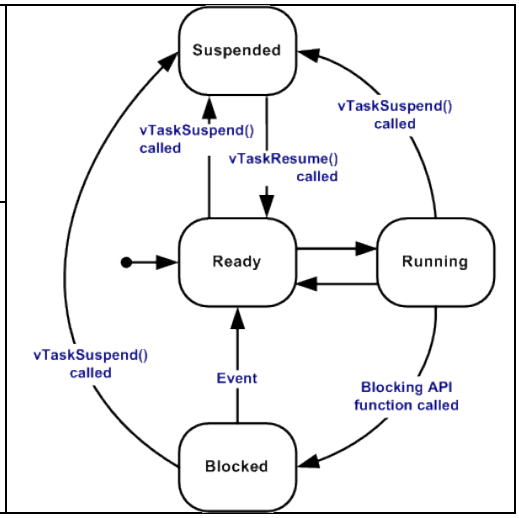
case 1: Task1 Priority 1, Task2 Priority 1	case 2: Task1 Priority 1, Task2 Priority 2
	

On case 1, both tasks have same priority, and after each time period, scheduler executes other task.
On case 2, Task1 is not executed, and we call it as a starvation.

3.4 Blocked and Suspended and Ready State (Not Running State)

Task has 4 states.

task states	Description	Related functions
Ready	Task on the ready queue.	
Running	Task that is executed	
Blocked	Blocked task that wait an event. ex: task tick.	vTaskDelay().
Suspended	No timeout period. To wait some resource	vTaskSuspend() and xTaskResume()

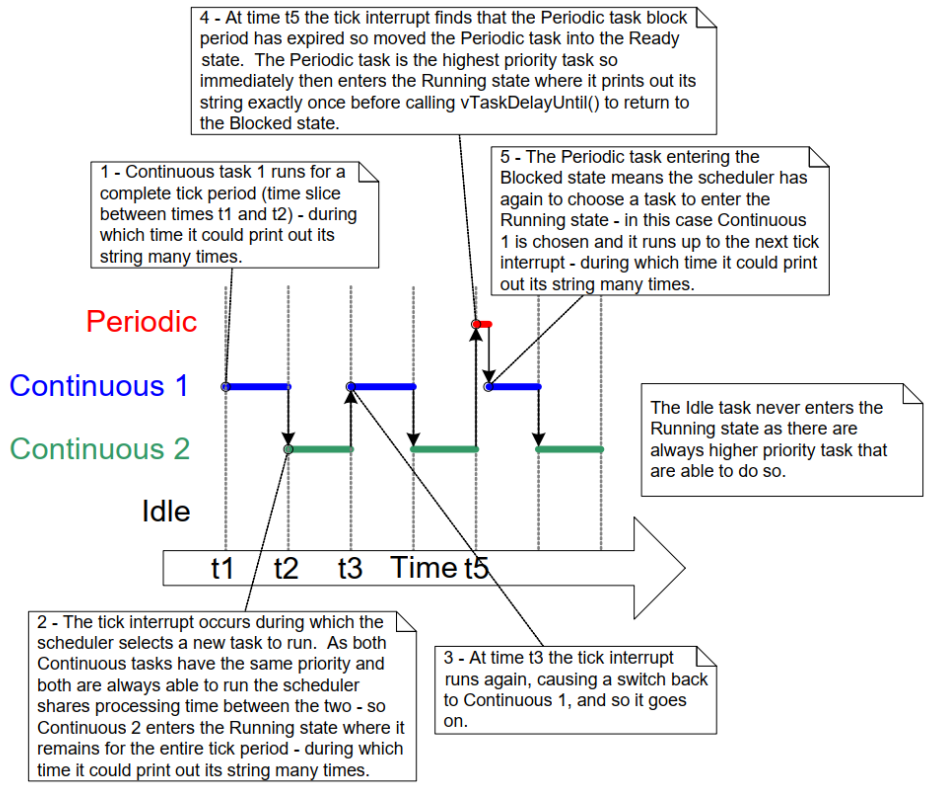
Blocked waits - Temporal (time-related) events - Synchronization event. queues, binary semaphores, counting semaphores, mutexes, recursive mutexes, event groups, stream buffers, message buffers, and direct to task notifications	
Suspended State Tasks in the Suspended state are not available to the scheduler. - vTaskSuspend(), vTaskResume() or xTaskResumeFromISR()	

3.5 Task Delay

vTaskDelay(xTicksToDelay)	How long the task will stay on Block state. xTicksToDelay - The number of tick interrupts. - Use pdMS_TO_TICKS() to convert ms to tick number. But the time at which the task leaves the blocked state is relative to the time at which vTaskDelay() was called. It can generate an accumulated issue.
vTaskDelayUnitl(pxRrevoiosWakeTime, xTimeIncrement)	It will **solve the accumulated issue**. Use when a **fixed execution period** is required. pxPreviousWakeTime - holds the time at which the task last left the Blocked state xTimeIncrement - The number of tick interrupts. This tick count which the calling task should be moved from the Blocked state.

Example: execute every 250ms exactly.

```
void vTaskFunc(void *param)
{
 char *pcTaskName;
 TickType_t xLastWakeTime;
 pcTaskName = (char *)param;
 xLastWakeTime = xTaskGetTickCount();

 for(;;) {
   vPrintLine(pcTaskName);
   vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS(250));
 }
}
```



3.6 Idle Task

The idle task is created automatically. Used when if no tasks are Ready state.
The Idle task has priority 0 (Lowest).
If you have priority 0 task, and do not want idle task waste CPU, use **configIDLE_SHOULD_YIELD** set 1.
The idle task is responsible for cleaning up kernel resources used by tasks that deleted themselves (vTaskDelete()).

We can add an application function on the idle task, (called as "Idle Task Hook Function" or callback). It has
- Executing low priority without RAM overhead.
- Measuring the amount of spare processing capacity
- Placing the processor into a low power mode (tick-less idle mode is better solution)

It need **configUSE_IDLE_HOOK** set 1, and the prototype is "void **vApplicationIdleHook**(void);"

3.7 Task Priority and Delete Task

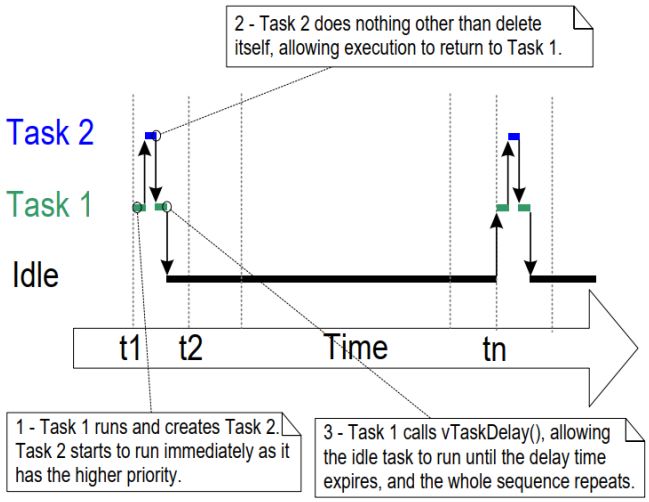
Change Task Priority	`void vTaskPrioritySet( TaskHandle_t xTask, uBaseType_t uxNewPriority);`
Get Task Priority	`uBaseType_t uxTaskPriorityGet( TaskHandle_t xTask);` `If NULL, the task can query its own priority.`
Delete Task	`void vTaskDelete( Taskhandle_t xTaskToDelete );` `Need INCLUDE_vTaskDelete 1`

Example. task1 create higher priority task2, and task2 delete itself.

```
int main( void )
{
  /* Create the first task at priority 1. */
  xTaskCreate( vTask1, "Task 1", 1000, NULL, 1, NULL );
  /* Start the scheduler so the task starts executing. */
  vTaskStartScheduler();

  /* main() should never reach here as the scheduler has been started. */
  for( ;; )
  {
  }
}
```

```
TaskHandle_t xTask2Handle = NULL;
void vTask1( void * pvParameters )
{
  const TickType_t xDelay100ms = pdMS_TO_TICKS( 100UL );
  for( ;; )
  {
    /* Print out the name of this task. */
    vPrintLine( "Task 1 is running" );
    /*
    * Create task 2 at a higher priority.
    * Pass the address of xTask2Handle as the pxCreatedTask parameter so
    * that xTaskCreate write the resulting task handle to that variable.
    */
    xTaskCreate( vTask2, "Task 2", 1000, NULL, 2, &xTask2Handle );
    /*
    * Task 2 has/had the higher priority. For Task 1 to reach here, Task 2
    * must have already executed and deleted itself.
    */
    vTaskDelay( xDelay100ms );
  }
}
```

```
void vTask2( void * pvParameters )
{
  /*
  * Task 2 immediately deletes itself upon starting.
  * To do this it could call vTaskDelete() using NULL as the parameter.
  * For demonstration purposes, it instead calls vTaskDelete() with its own
  * task handle.
  */
  vPrintLine( "Task 2 is running and about to delete itself" );
  vTaskDelete( xTask2Handle );
}
```



3.8 TLS and Reentrancy

TLS (Thread Local Storage) allows storing arbitrary data in TCB (Task Control Block), example is errno.
A reentrant function is a function which can safely run from multiple threads without any side effects of critical section. FreeRTOS support reentrant API with below two pre-built C Runtime libraries.

for newlib	configUSE_NEWLIB_REENTRANT
for picolibc	configUSE_PICOLIBC_TLS

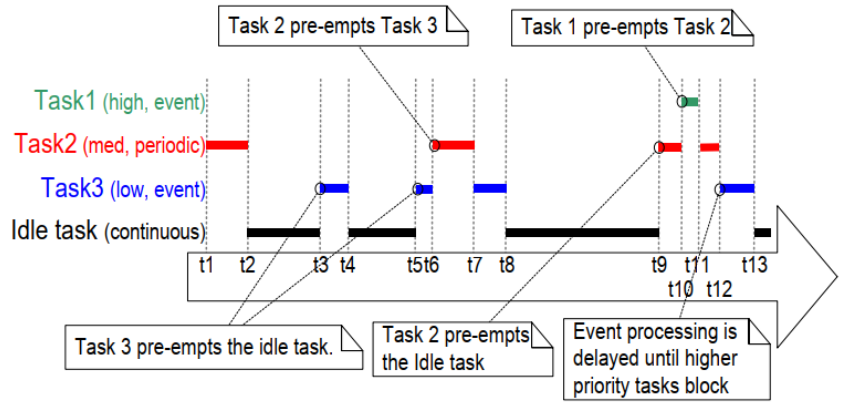
Developer can define a set of application specific pointers to be included in the task control block.
configNUM_THREAD_LOCAL_STORAGE_POINTERS to a non-zero

pvTaskGetThreadLocalStoragePointer() vTaskSetThreadLocalStoragePointer()

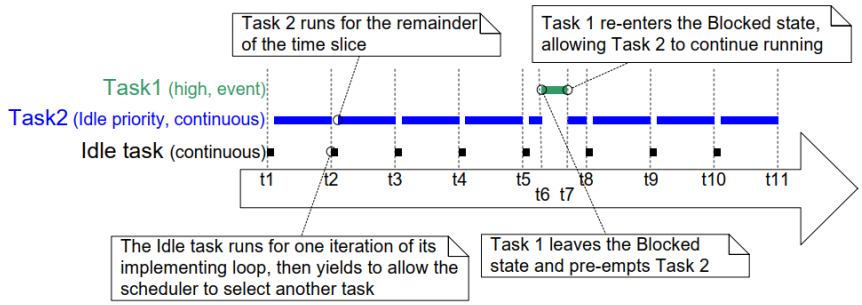
3.9 Scheduling Algorithm

Most popular scheduling algorithm is **Fixed Priority Preemptive Scheduling with Time Slicing**.

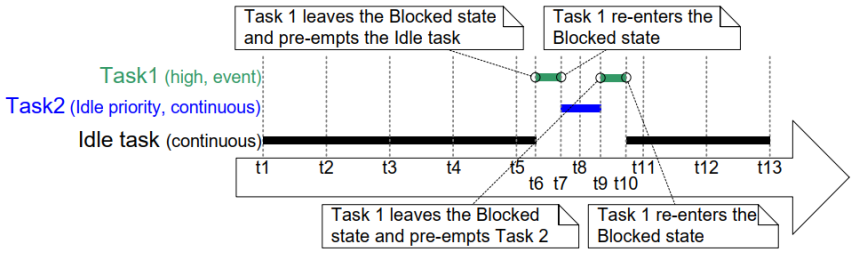
	Description
Fixed Priority	Do not change the priority assigned to the tasks being scheduled. Do not prevent the tasks themselves from changing their own priority or that of other tasks.
Preemptive	immediately 'preempt' the *Running* state task if a task that has a priority higher than the *Running* state task enters the *Ready* state
Time Slicing	between tasks of equal priority



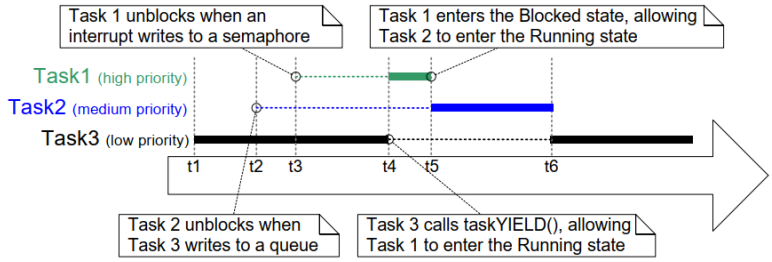
configIDLE_SHOULD_YIELD is set to 1 then the Idle task yields (voluntarily gives up whatever remains of its allocated time slice) on each iteration of its loop if there are other Idle priority tasks in the *Ready* state.



It does not use time slicing to share processing time between tasks of equal priority.



Cooperative Scheduling is a context switch only occurs when the Running state task enters the *Blocked* state, or the *Running* state task explicitly yields (manually requests a re-schedule) by calling taskYIELD(). Tasks are never preempted, so time slicing cannot be used.



3.10 Task Control Block

typedef struct tskTaskControlBlock {	
volatile portSTACK_TYPE *px**TopOfStack**;	Points to the location of the last item placed on the tasks stack. THIS MUST BE THE FIRST MEMBER OF THE STRUCT.
xListItem x**GenericListItem**;	List item used to place the TCB in ready and blocked queues.
xListItem x**EventListItem**;	List item used to place the TCB in event lists.
unsigned portBASE_TYPE ux**Priority**;	The priority of the task where 0 is the lowest priority.
portSTACK_TYPE *px**Stack**;	Points to the start of the stack.
signed char pc**TaskName**[configMAX_TASK_NAME_LEN];	Descriptive name given to the task when created. Facilitates debugging only.
#if (portSTACK_GROWTH > 0) portSTACK_TYPE *pxEndOfStack; #endif	Used for stack overflow checking on architectures where the stack grows up from low memory.
#if (configUSE_MUTEXES == 1) unsigned portBASE_TYPE ux**BasePriority**; #endif	The priority last assigned to the task - used by the priority inheritance mechanism.
} tskTCB;	

3.11 Inter Task Communications

	Description
Queue	a task-to-task, task-to-interrupt, and interrupt-to-task communication mechanism.
Binary Semaphores	Signaling. Binary Semaphore is better for synchronization. Mutex is better for mutual execution.
Counting Semaphores	Signaling. For counting events
Mutex	Mutual Exclusion concept. To control access to a resource that is shared between two or more tasks. Acts like a token that is used to guard a resource. Priority Inversion, Priority Inheritance
Event Group	Event groups allow a task to wait in the Blocked state for a combination of one of more events to occur. Event groups unblock all the tasks that were waiting for the same event, or combination of events, when the event occurs.
task notification	An efficient mechanism allowing one task to directly notify another task.

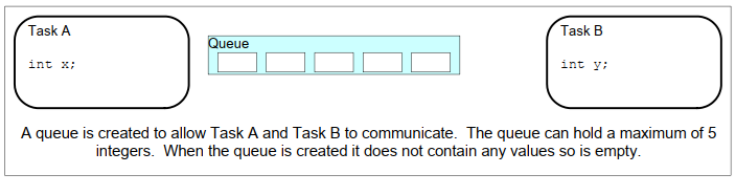
4 Queue

Queue can hold a finite number of fixed size data. Write at end (tail), read from front (head).
Two ways behaviors

Queue by copy	normal. simple.
Queue by reference	queue only holds pointers to the data sent to the queue, not the data itself.

Queue can be task-to-task, task-to-interrupt, and interrupt-to-task communication mechanism.

	function	ISR
Create	xQueueCreate() xQueueCreateStatic()	
Send	xQueueSendToBack() (or xQueueSend()), xQueueSendToFront()	xQueueSendToFrontFromISR(), xQueueSendToBackFromISR().
Receive	xQueueReceive()	xQueueReceiveFromISR()
Wait	uxQueueMessageWaiting(),	xQueueMessageWaitingFromISR()



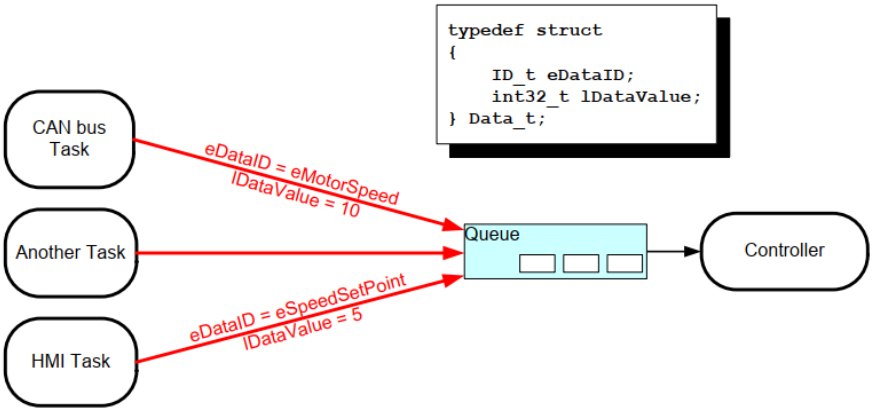
..send “20”

If queue is empty, reader is blocking. If queue is full, writer is blocking.

If data is large, use a pointer. In this case, we need,

The owner of the RAM being pointed to is clearly defined. Do not modify the memory contents simultaneously. Ideally, only the sending task should be permitted to access the memory before the pointer is sent to the queue, and only the receiving task should be permitted to access the memory after the pointer has been received from the queue.
The RAM being pointed to remains valid. If memory dynamically allocated, exactly one task should be responsible for freeing the memory.

4.1 Receiving Data from Multiple Source



Multiple writers are common, but much less common for a queue to have multiple readers.
In the case of multiple writers, semaphore can be added on the queue set.

	Function
Create set	xQueueCreateSet()
Add queue or semaphore	xQueueAddToSet()
Read a queue handle	xQueueSelectFromSet()

4.2 Mailbox (Queue Length 1)

The term mailbox is used to refer to a queue that has a length of one.

```
typedef struct xExampleStructure
{
  TickType_t xTimeStamp;
  uint32_t ulValue;
} Example_t;
QueueHandle_t xMailbox;

void vAFunction(void)
{
  xMailbox = xQueueCreate(1, sizeof(Example_t));
}
```

If queue length is 1, we can use,

	Function
Overwrite	xQueueOverwrite(), xQueueOverwriteFromISR(). if the queue is already full, then xQueueOverwrite() overwrites data that is already in the queue.
read without remove.	xQueuePeek() receives (reads) an item from a queue *without* removing the item from the queue.

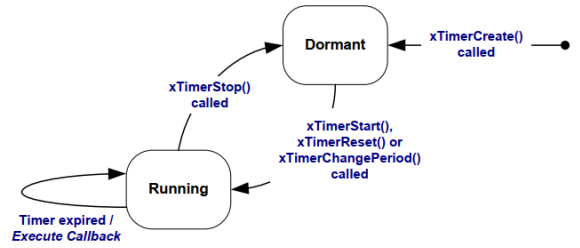
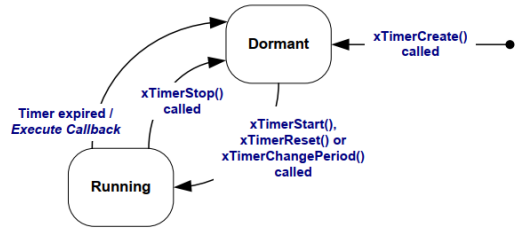
5 Software Timer

Software timers are used to schedule the execution of a function at a set time in the future, or periodically with a fixed frequency.

	Functions
callback	ATimerCallback(). Software timer callback function, should be kept short, and must not enter the Blocked state. Software timer callback functions must not call FreeRTOS API functions that will result in the calling task entering the Blocked state, as to do so will result in the daemon task entering the Blocked state.
Create	xTimerCreate(), xTimerCreateStatic()
Start	xTimerStart(),
delete	xTimerDelete()
Set ID	vTimerSetTimerID()
Get ID	pvTimerGetTimerID()
Changing Period Timer	xTimerChangePeriod()
Reset	xTimerReset()

5.1 Software Timer status.

Auto-reload or one-shot software timer

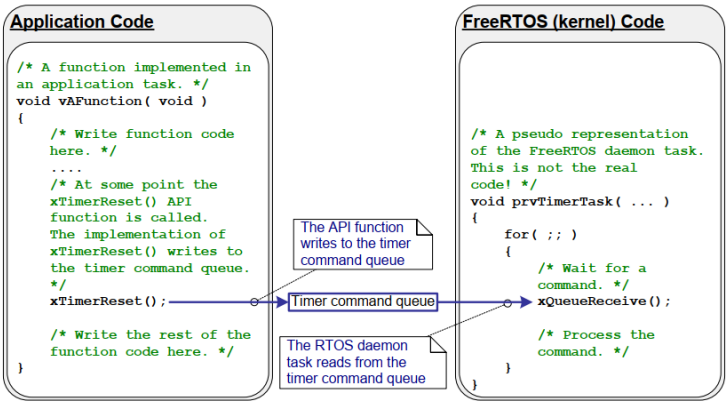
Auto-reload	One-shot
	

5.2 RTOS Daemon task (or timer service task)

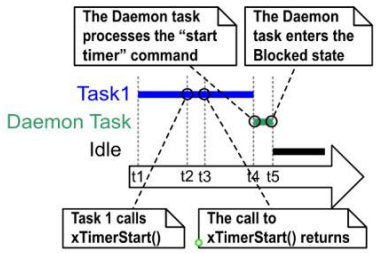
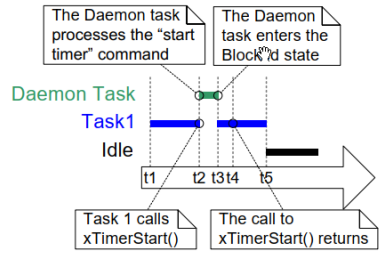
All software timer callback functions execute in the context of the same RTOS daemon (or 'timer service') task.

The daemon task is a standard FreeRTOS task that is created automatically when the scheduler is started.

Timer Command Queue is automatically created, and can be ‘start’, ‘stop’ and ‘reset’



Daemon Task Scheduling

when the priority of a task calling xTimerStart() is above the priority of the daemon task	when the priority of a task calling xTimerStart() is below the priority of the daemon task
	

6 Interrupt Management

Embedded real-time systems have to take actions in response to events. How should the event be detected? Interrupts are normally used. It is normally desirable to keep each ISR as short as possible. It is important to draw a distinction between the priority of a task, and the priority of an interrupt, the lowest priority interrupt will interrupt the highest priority task.

6.1 Interrupt Safe API.

ISR uses a separate Interrupt Safe API. Because ISR must not move to Block status.
API for ISR has “FromISR” prefix.

6.2 xHigherPriorityTaskWoken parameter

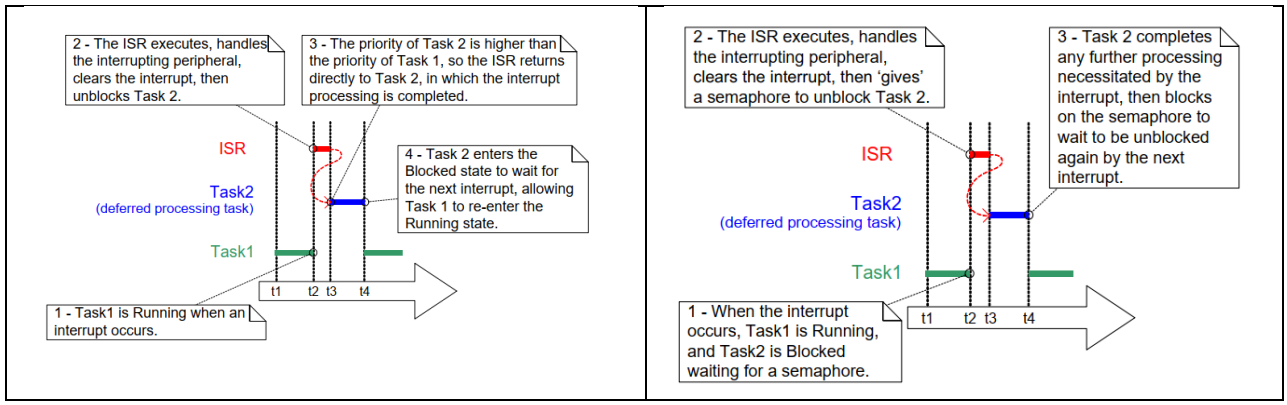
Within ISR, context switch is not happened.
When ISR is finished, control can be needed to go to other higher task, not the interrupted task.
xHigherPriorityTaskWoken is used to decide the context switch can be or should not be happened after ISR.
Without this mechanism, once an interrupt concludes, the system would return to the previously executing lower-priority task and have to wait until the next Tick Interrupt (timer interrupt) occurs for scheduling to take place.

example:

```
void vExampleInterruptHandler(void) {
    // 1. Init as pdFALSE on Waken flag.
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    // 2. send Waken flag to FromISR() function.
    // If it is needed the context switch to higher task, it changes as pdTRUE.
    xQueueSendFromISR(xQueue, &data, &xHigherPriorityTaskWoken);

    // 3. At the end, Request Context switch,
    // if it was pdTRUE, it will jump to high priority task.
    //if(xHigherPriorityTaskWoken == pdTRUE)
      portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    /* or use portEND_SWITCHING_ISR() */
}
```

6.3 Deferred Interrupt Processing

interrupt processing in a high priority task	Using a binary semaphore to implement deferred interrupt processing



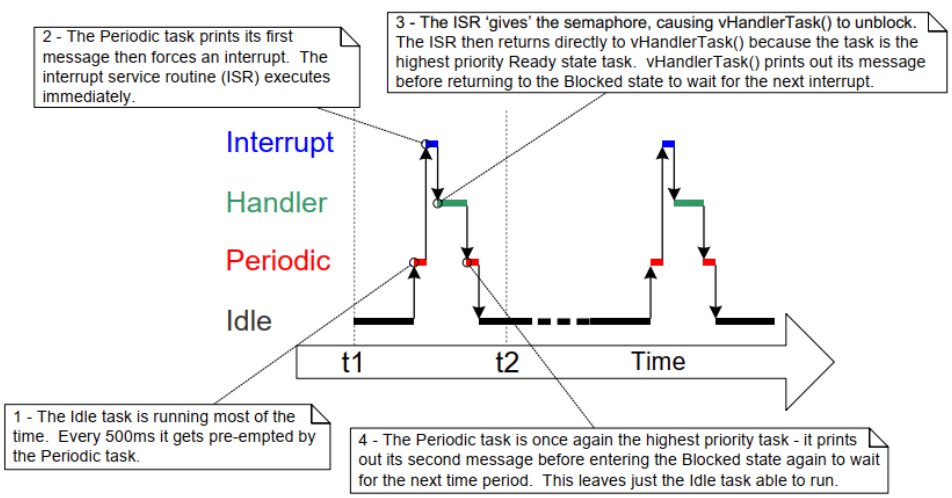
Semaphore is for synchronization. The deferred interrupt processing is good example for semaphore.

6.4 Binary Semaphore

	Functions
create	xSemaphaoreCreateBinary(void)
taking	xSemaphoreTake(xSemaphore, xTicksToWait)
Given	xSemaphoreGiveFromISR()

Example: Using a binary semaphore to synchronize a task with an interrupt

A simple periodic task is used to generate a software interrupt every 500 milliseconds.




```
#define mainINTERRUPT_NUMBER 3
static void vPeriodicTask( void *pvParameters )
{
  const TickType_t xDelay500ms = pdMS_TO_TICKS( 500UL );
  for( ;; )
  {
    vTaskDelay( xDelay500ms );

    /* Generate the interrupt, printing a message both before and after the interrupt has
       been generated, so the sequence of execution is evident from the output.
```

```
        The syntax used to generate a software interrupt is dependent on the FreeRTOS port
        being used. The syntax used below can only be used with the FreeRTOS Windows port,
        in which such interrupts are only simulated. */
    vPrintString( "Periodic task - About to generate an interrupt.\r\n" );
    vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
    vPrintString( "Periodic task - Interrupt generated.\r\n\r\n\r\n" );
  }
}
```

```
static void vHandlerTask( void *pvParameters )
{
  for( ;; )
  {
    xSemaphoreTake( xBinarySemaphore, portMAX_DELAY );
    vPrintString( "Handler task - Processing event.\r\n" );
  }
}
```

```
static uint32_t ulExampleInterruptHandler( void )
{
  BaseType_t xHigherPriorityTaskWoken;
  xHigherPriorityTaskWoken = pdFALSE;
  xSemaphoreGiveFromISR( xBinarySemaphore, &xHigherPriorityTaskWoken );
  portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

```
int main( void )
{
  xBinarySemaphore = xSemaphoreCreateBinary();
  if( xBinarySemaphore != NULL )
  {
    xTaskCreate( vHandlerTask, "Handler", 1000, NULL, 3, NULL );
    xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, 1, NULL );
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulExampleInterruptHandler );
    vTaskStartScheduler();
  }
  for( ;; );
}
```

The other use case for semaphore is UART RX task, because it does not know when it has a received data.

```
#include<stdlib.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

xSemaphoreHandle xBinarySemaphore;
static void vUARTReceiveHandlerTask( void *pvParameters )
{
  /* xMaxExpectedBlockTime holds the maximum time expected between two interrupts. */
  const TickType_t xMaxExpectedBlockTime = pdMS_TO_TICKS( 500 );
  /* As per most tasks, this task is implemented within an infinite loop. */
  for( ;; )
  {
    /* The semaphore is 'given' by the UART's receive (Rx) interrupt.
       Wait a maximum of xMaxExpectedBlockTime ticks for the next interrupt. */
    if( xSemaphoreTake( xBinarySemaphore, xMaxExpectedBlockTime ) == pdPASS)
    {
```

```
      /* The semaphore was obtained. Process ALL pending Rx events before
         calling xSemaphoreTake() again. Each Rx event will have placed a
         character in the UART's receive FIFO, and UART_RxCount() is
         assumed to return the number of characters in the FIFO. */
      while( UART_RxCount() > 0 )
      {
        /* UART_ProcessNextRxEvent() is assumed to process one Rx
          character, reducing the number of characters in the FIFO by 1. */
        UART_ProcessNextRxEvent();
      }
      /* No more Rx events are pending (there are no more characters in
       the FIFO), so loop back and call xSemaphoreTake() to wait for
       the next interrupt. Any interrupts occurring between this point
       in the code and the call to xSemaphoreTake() will be latched in
       the semaphore, so will not be lost. */
    }
    else
    {
      /* An event was not received within the expected time. Check for,
      and if necessary clear, any error conditions in the UART that
      might be preventing the UART from generating any more interrupts. */
      UART_ClearErrors();
    }
  }
}
```

```
void initpll(void)
{
  PLL0CON=0x01;
  PLL0CFG=0x24;
  PLL0FEED=0xAA;
  PLL0FEED=0x55;
  while(!(PLL0STAT&1<<10));
  PLL0CON=0x03;
  PLL0FEED=0xAA;
  PLL0FEED=0x55;
  VPBDIV=0x01;
}

void rx1(void)__irq
{
  unsigned char r;
  static portBASE_TYPE interrupttask; /* BaseType_t xHigherPriorityTaskWoken */
  interrupttask=pdFALSE;

  if(U0IIR&0x04) {
    r=U0RBR;
    sends_serial("Received Data : ");
    U0THR=r;
    while(!(U0LSR&0x20));
    sends_serial("\r\n");
    sends_serial("Going to give semaphore");
    sendsserial("\r\n");
    xSemaphoreGiveFromISR(xBinarySemaphore,&interrupttask);
    sends_serial("Semaphore has given");
    sends_serial("\r\n");
  }
  VICVectAddr = 0x00;
}
//---------------------
// add rx1 ISR
void init_serial()
```

```
{
  VPBDIV=0x01;
  PINSEL0|=0x5;
  U0LCR=0x83;
  U0DLL=136;   //u0dll=60Mhz/(16*9600)
  U0DLM=1;
  U0LCR=0x03;
  U0TER=(1<<7);
  U0IER=0x01;  //enables uart receive identification interrupt
  VICIntSelect=0x0000;     //selecting as irq
  VICIntEnable|=0x0040;   //enable uart1
  VICVectAddr2=(unsigned long int)rx1;  //assigning address @@@
  VICVectCntl2=0x26;  //need to assign slot no of interrupt source which has been activated
}

//-----------------------------------
// Tx
//-----------------------------------
void send_serial(unsigned char dat)
{
  U0THR=dat;
  while((U0LSR&(1<<5))==0);
}

void sends_serial(unsigned char *sdat)
{
  while(*sdat) {
    send_serial(*sdat++);
  }
}
```

```
int main()
{
  vSemaphoreCreateBinary(xBinarySemaphore);
  init_serial();
  xSemaphoreTake(xBinarySemaphore,portMAX_DELAY);
  xTaskCreate(vUARTReceiveHandlerTask,"intertask",128,NULL,1,NULL);
  vTaskStartScheduler();
  while(1);
}
```

6.5 Counting Semaphores

- For counting events
- For Resource management

	Function
Create	xSemaphoreCreateCounting(ux**MaxCount**, uxInitCount);

Use same ‘take’, ‘given’ functions that we used on the binary semaphore.

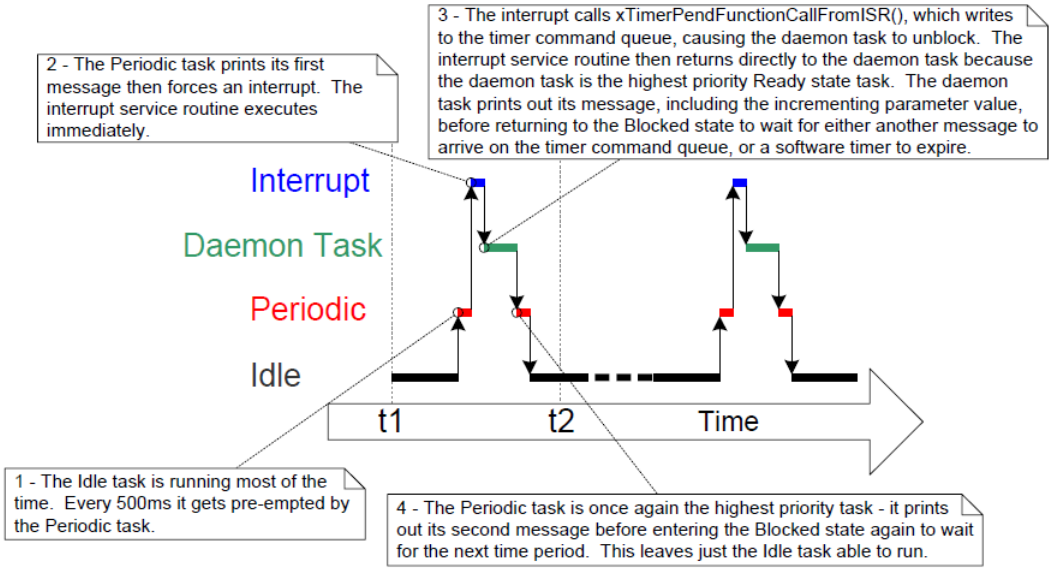
6.6 Deferring Work to the RTOS Daemon Task.

The deferred interrupt handling examples presented so far have required the application writer to create a task for each interrupt that uses the **deferred processing** technique.

The daemon task was originally called the timer service task because it was originally only used to execute software timer callback functions.

If use xTimerPendFunctionCallFromISR() API function to defer interrupt processing to the RTOS daemon task, which removes the need to create a separate task for each interrupt. Deferring interrupt processing to the daemon task is called **'centralized deferred interrupt processing'**. It uses the same timer command queue to send an 'execute function' command to the daemon task.

use the same timer command queue to send an 'execute function' command to the daemon task on software timer callback function.	xTimerPendFunctionCallFromISR(PendedFunction_t xFunctionToPend, void *pvParameter1, uint32_t ulParameter2, BaseType_t *pxHigherPriorityTaskWoken);
The prototype to which a function passed in the xFunctionToPend parameter of xTimerPendFunctionCallFromISR() must conform	void vPendableFunction(void *pvParameter1, uint32_t ulParameter2);



6.7 Queue within ISR

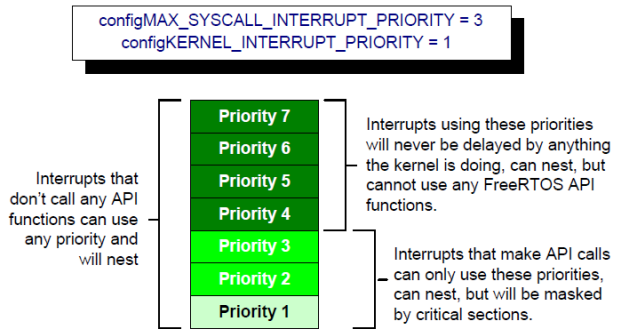
normal	Queue within ISR
xQueueSendToFront()	xQueueSendToFrontFromISR()
xQueueSendToBack()	xQueueSendToBackFromISR()
xQueueReceive()	xQueueReceiveFromISR()

6.8 Interrupt Nesting

Hardware decides when an ISR will execute, whereas software decides when a task will execute.
Each interrupt source has a numeric priority, and a logical priority:

Numeric priority	The numeric priority is simply the number assigned to the interrupt priority.
Logical priority	An interrupt's logical priority describes that interrupt's precedence over other interrupts.

	If two interrupts of differing priority occur at the same time, then the processor will execute the ISR that has the higher logical priority. An interrupt can interrupt (nest with) any interrupt that has a lower logical priority, but an interrupt cannot interrupt (nest with) any interrupt that has an equal or higher logical priority.



Typically, functionality that requires very strict timing accuracy (motor control, for example) would use a priority above configMAX_SYSCALL_INTERRUPT_PRIORITY to ensure the scheduler does not introduce jitter into the interrupt response time.

ARM Cortex-M

The ARM Cortex cores, and ARM Generic Interrupt Controllers (GICs), use numerically *low* priority numbers to represent logically *high* priority interrupts.
The Cortex-M interrupt controller allows a maximum of eight bits to be used to specify each interrupt priority, making 255 the lowest possible priority. Zero is the highest priority. Cortex-M Microcontrollers normally only implement a subset of the eight possible bits. It is only the most significant bits of the byte that can be used.

7 Resource Management

7.1 Mutual Exclusion

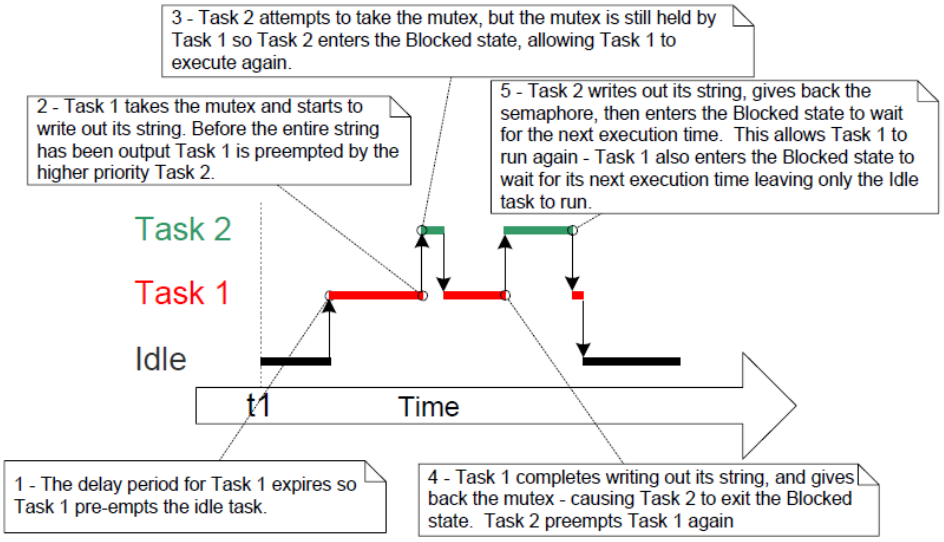
The goal is to ensure that, once a task starts to access a shared resource that is not re-entrant and not thread-safe, the same task has exclusive access to the resource until the resource has been returned to a consistent state.

7.2 Critical Section and Suspend Scheduler

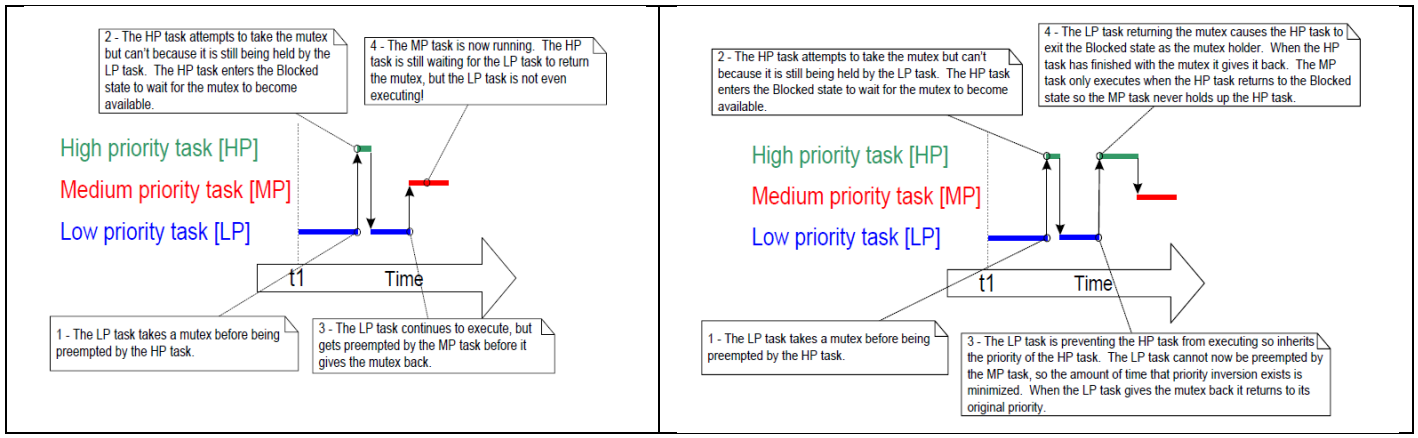
	function	ISR
enter	taskENTER_CRITICAL()	taskENTER_CRITICAL_FROM_ISR()
exit	taskEXIT_CRITICAL()	taskEXIT_CRITICAL_FROM_ISR()
suspend	vTaskSuspendAll()	
resume	xTaskResumeAll()	

7.3 Mutex

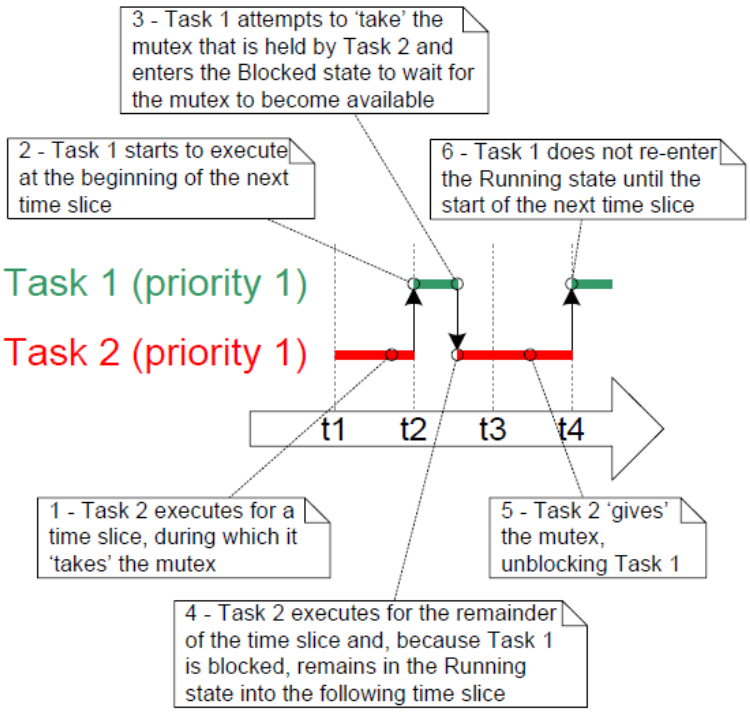
	function	Recursive Mutex
For mutex	xSemaphoreCreateMutex() xSemaphoreCreateMutexStatic()	xSemaphoreCreateRecursiveMutex()
common	xSemaphoreTake() xSemaphoreGive()	xSemaphoreTakeRecursive() xSemaphoreGiveRecursive()



Priority Inversion	Priority Inheritance



Mutex and Task Scheduling



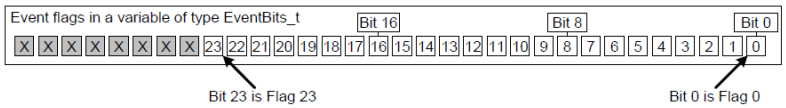
7.4 Gatekeeper Tasks

A gatekeeper task is a task that has sole ownership of a resource.

8 Event Group

semaphore and queue	Event Group
They allow a task to wait in the Blocked state for a single event to occur. They unblock a single task when the event occurs. The task that is unblocked is the highest priority task that was waiting for the event.	Event groups allow a task to wait in the Blocked state for a combination of one of more events to occur. Event groups unblock all the tasks that were waiting for the same event, or combination of events, when the event occurs.

EventBits_t data type



		ISR
create	xEventGroupCreate() , xEventGroupCreateStatic()	
sets one or more bits in an event group	xEventGroupSetBits()	xEventGroupSetBitsFromISR()
wait	xEventGroupWaitBits()	
Get	xEventGroupGetStaticBuffer()	
sync	xEventGroupSync()	

8.1 Task Synchronization Using an Event Group

The demonstration shares a TCP socket between two tasks; one task sends data to the socket, and a different task receives data from the same socket. If either of the two tasks wishes to close the socket, then it must inform the other task of its intent, and then wait for the other task to stop using the socket before proceeding. The scenario where it is the task that sends data to the socket that wishes to close the socket is demonstrated by the pseudo code.

Pseudo code for two tasks that synchronize with each other to ensure a shared TCP socket is no longer in use by either task before the socket is closed

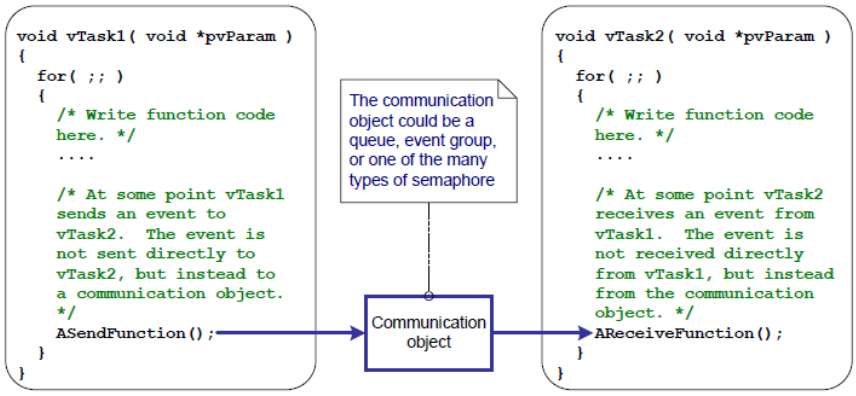
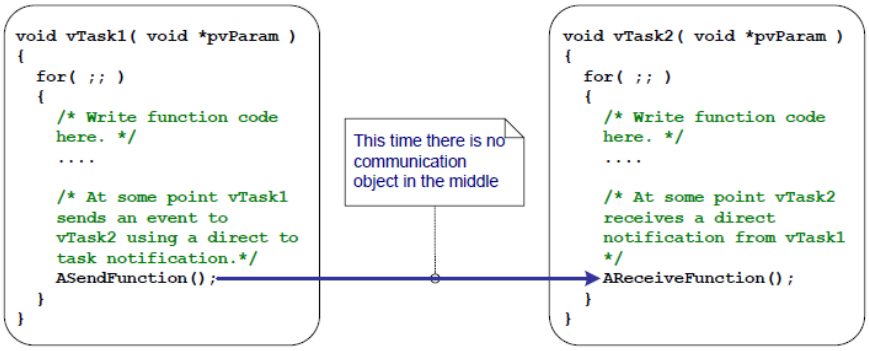
```
void SocketTxTask( void *pvParameters )
{
  xSocket_t xSocket;
  uint32_t ulTxCount = 0UL;
  for( ;; )
  {
    /* Create a new socket. This task will send to this socket, and another
       task will receive from this socket. */
    xSocket = FreeRTOS_socket( ... );
    /* Connect the socket. */
    FreeRTOS_connect( xSocket, ... );
    /* Use a queue to send the socket to the task that receives data. */
    xQueueSend( xSocketPassingQueue, &xSocket, portMAX_DELAY );
    /* Send 1000 messages to the socket before closing the socket. */
    for( ulTxCount = 0; ulTxCount < 1000; ulTxCount++ )
    {
```

```
      if( FreeRTOS_send( xSocket, ... ) < 0 )
      {
        /* Unexpected error
        - exit the loop, after which the socket will be closed. */
        break;
      }
    }
    /* Let the Rx task know the Tx task wants to close the socket. */
    TxTaskWantsToCloseSocket();

    /* This is the Tx task's synchronization point. The Tx task waits here
       for the Rx task to reach its synchronization point. The Rx task will
       only reach its synchronization point when it is no longer using the
      socket, and the socket can be closed safely. */
    xEventGroupSync( ... );

    /* Neither task is using the socket.
       Shut down the connection, then close the socket. */
    FreeRTOS_shutdown( xSocket, ... );
    WaitForSocketToDisconnect();
    FreeRTOS_closesocket( xSocket );
  }
}
```

```
void SocketRxTask( void *pvParameters )
{
  xSocket_t xSocket;
  for( ;; )
  {
    /* Wait to receive a socket that was created and connected by the Tx task. */
    xQueueReceive( xSocketPassingQueue, &xSocket, portMAX_DELAY );
    /* Keep receiving from the socket until the Tx task wants to close the socket. */
    while( TxTaskWantsToCloseSocket() == pdFALSE )
    {
      /* Receive then process data. */
      FreeRTOS_recv( xSocket, ... );
      ProcessReceivedData();
    }
    /* This is the Rx task's synchronization point
     - it only reaches here when it is no longer using the socket,
       and it is therefore safe for the Tx task to close the socket. */
    xEventGroupSync( ... );
  }
}
```

9 Task Notifications

Task notifications are an efficient mechanism allowing one task to directly notify another task.
(Others are called as 'Communication objects')

Common communication using an object (queue, event group, semaphores)	
Direct to task Communication.	

When configUSE_TASK_NOTIFICATIONS is set to 1, each task has at least one **'Notification State'**, which can be either **'Pending'** or **'Not-Pending'**, and a **'Notification Value'**, which is a 32-bit unsigned integer. When a task receives a notification, its notification state is set to pending. When a task reads its notification value, its notification state is set to not-pending.

Benefits

fast	Task Notification is significantly faster than using a queue, semaphore or event group to perform an equivalent operation.
use less RAM	Task Notification requires significantly less RAM than using a queue, semaphore or event group to perform an equivalent operation

Limitation

Sending an event or data to an ISR	Task notifications can be used to send events and data from an ISR to a task, but they cannot be used to send events or data from a

	task to an ISR.
Enabling more than one receiving task	Task notifications are sent directly to the receiving task, so they can only be processed by the task to which the notification is sent
Buffering multiple data items	Task notifications send data to a task by updating the receiving task's notification value. A task's notification value can only hold one value at a time.
Broadcasting to more than one task	Task notifications are sent directly to the receiving task, so can only be processed by the receiving task.
Waiting in the blocked state for a send to complete	If a task attempts to send a task notification to a task that already has a notification pending, then it is not possible for the sending task to wait in the Blocked state for the receiving task to reset its notification state.

		indexed	ISR
send receive	xTaskNotify() xTaskNotifyWait()	xTaskNotifyIndexed() xTaskNotifyWaitIndexed()	xTaskNotifyFromISR xTaskNotifyIndexedFromISR
simpler	xTaskNotifyGive() vTaskNotifyTake()	xTaskNotifyGiveIndexed() vTaskNotifyTakeIndexed()	vTaskNotifyGiveFromISR vTaskNotifyGiveIndexedFromISR
stateClear ValueClear	xTaskNotifyStateClear ulTaskNotifyValueClear	xTaskNotifyStateClearIndexed ulTaskNotifyValueClearIndexed	
AndQuery	xTaskNotifyAndQuery	xTaskNotifyAndQueryIndexed	xTaskNotifyAndQueryFromISR xTaskNotifyAndQueryIndexedFromISR

9.1 ADC example

This section demonstrates how to use xTaskNotifyFromISR() to send data with a task notification event.

- It is assumed an ADC conversion is started at least every 50 milliseconds.
- ADC_ConversionEndISR() is the interrupt service routine for the ADC's conversion end interrupt, which is the interrupt that executes each time a new ADC value is available.
- The task implemented by vADCTask() processes each value generated by the ADC. It is assumed the task's handle was stored in **xADCTaskToNotify** when the task was created.
- ADC_ConversionEndISR() uses xTaskNotifyFromISR() with the eAction parameter set to eSetValueWithoutOverwrite to send a task notification to the vADCTask() task, and write the result of the ADC conversion into the task's notification value.
- The vADCTask() task uses xTaskNotifyWait() to wait to be notified that a new ADC value is available, and to retrieve the result of the ADC conversion from its notification value.

```
/* A task that uses an ADC. */
void vADCTask( void *pvParameters )
{
  uint32_t ulADCValue;
  BaseType_t xResult;

  /* The rate at which ADC conversions are triggered. */
  const TickType_t xADCConversionFrequency = pdMS_TO_TICKS( 50 ); /* 50ms */
  for( ;; )
  {
    /* Wait for the next ADC conversion result. */
```

```
    xResult = xTaskNotifyWait(
      /* The new ADC value will overwrite the old value,
       so there is no need to clear any bits before waiting for the new notification value. */
      0,
      /* Future ADC values will overwrite the existing value,
         so there is no need to clear any bits before exiting xTaskNotifyWait(). */
      0,
      /* The address of the variable into which the task's notification value
        (which holds the latest ADC conversion result) will be copied. */
      &ulADCValue,
      /* A new ADC value should be received every xADCConversionFrequency ticks. */
      xADCConversionFrequency * 2 );
    if( xResult == pdPASS )
    {
      /* A new ADC value was received. Process it now. */
      ProcessADCResult( ulADCValue );
    }
    else
    {
      /* The call to xTaskNotifyWait() did not return within the expected time,
         something must be wrong with the input that triggers the ADC conversion,
         or with the ADC itself. Handle the error here. */
    }
  }
}
```

```
/* The interrupt service routine that executes each time an ADC conversion completes. */
void ADC_ConversionEndISR( xADC *pxADCInstance )
{
  uint32_t ulConversionResult;
  BaseType_t xHigherPriorityTaskWoken = pdFALSE, xResult;

  /* Read the new ADC value and clear the interrupt. */
  ulConversionResult = ADC_low_level_read( pxADCInstance );

  /* Send a notification, and the ADC conversion result, directly to vADCTask(). */
  xResult = xTaskNotifyFromISR( xADCTaskToNotify, /* xTaskToNotify parameter */
                                ulConversionResult, /* ulValue parameter */
                                eSetValueWithoutOverwrite, /* eAction parameter. */
                                &xHigherPriorityTaskWoken );

  /* If the call to xTaskNotifyFromISR() returns pdFAIL then the task is not keeping up with
     the rate at which ADC values are being generated. */
  configASSERT( xResult == pdPASS );
  portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

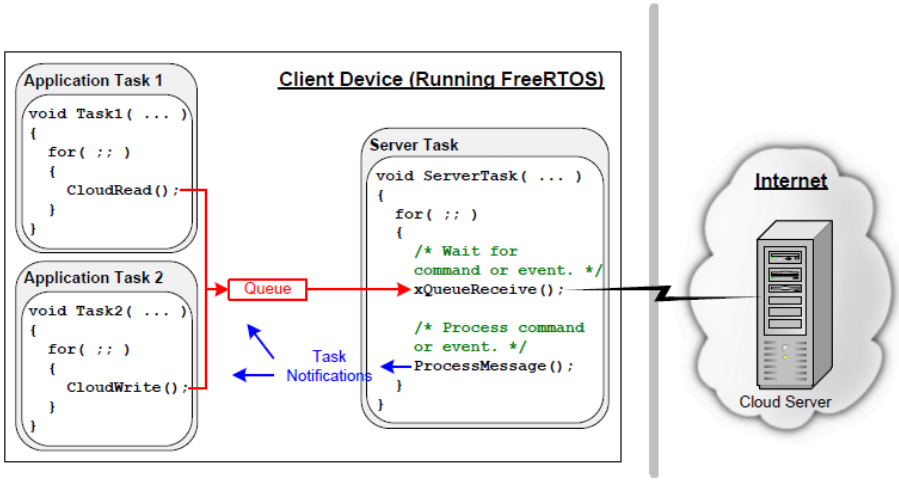
Pseudo code demonstrating how a task notification can be used to pass a value to a task.

9.2 Task Notifications Used Directly Within an Application

The application communicates across a slow internet connection to send data to, and request data from, a remote data server. From here on, the remote data server is referred to as the ***cloud server***.

After requesting data from the cloud server, the requesting task must wait in the Blocked state **for the requested data to be received**.

After sending data to the cloud server, the sending task must wait in the Blocked state **for an acknowledgement** that the cloud server received the data correctly.




```
typedef enum CloudOperations
{
  eRead, /* Send data to the cloud server. */
  eWrite /* Receive data from the cloud server. */
} Operation_t;

typedef struct CloudCommand
{
  Operation_t eOperation; /* The operation to perform (read or write). */
  uint32_t ulDataID; /* Identifies the data being read or written. */
  uint32_t ulDataValue; /* Only used when writing data to the cloud server. */
  TaskHandle_t xTaskToNotify;/* The handle of the task performing the operation. */
} CloudCommand_t;
```

Pseudo Read

```
/* ulDataID identifies the data to read. pulValue holds the address of the
variable into which the data received from the cloud server is to be written.
*/
BaseType_t CloudRead( uint32_t ulDataID, uint32_t *pulValue )
{
  CloudCommand_t xRequest;
  BaseType_t xReturn;

  /* Set the CloudCommand_t structure members to be correct for this read request. */
  xRequest.eOperation = eRead; /* This is a request to read data. */
  xRequest.ulDataID = ulDataID; /* A code that identifies the data to read. */
  xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle(); /* Handle of the calling task. */

  /* Ensure there are no notifications already pending by reading the notification value
    with a block time of 0, then send the structure to the server task. */
  xTaskNotifyWait( 0, 0, NULL, 0 );
  xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

  /* Wait for a notification from the server task. The server task writes the value received
    from the cloud server directly into this task's notification value, so there is no need to
    clear any bits in the notification value on entry to or exit from the xTaskNotifyWait()
    function. The received value is written to *pulValue, so pulValue is passed as the address
    to which the notification value is written. */
  xReturn = xTaskNotifyWait( 0, /* No bits cleared on entry */
              0, /* No bits to clear on exit */
```

```
                pulValue, /* Notification value into *pulValue */
                pdMS_TO_TICKS( 250 ) ); /* Wait 250ms maximum */
  /* If xReturn is pdPASS, then the value was obtained. If xReturn is pdFAIL,
     then the request timed out. */
  return xReturn;
}
```

Server task processing a Read Request

```
void ServerTask( void *pvParameters )
{
  CloudCommand_t xCommand;
  uint32_t ulReceivedValue;
  for( ;; )
  {
    /* Wait for the next CloudCommand_t structure to be received from a task */
    xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );
    switch( xCommand.eOperation ) /* Was it a read or write request? */
    {
    case eRead:
      /* Obtain the requested data item from the remote cloud server */
      ulReceivedValue = GetCloudData( xCommand.ulDataID );

      /* Call xTaskNotify() to send both a notification and the value received
        from the cloud server to the task that made the request.
        The handle of the task is obtained from the CloudCommand_t structure. */
      xTaskNotify( xCommand.xTaskToNotify, /* The task's handle is in the structure */
        ulReceivedValue, /* Cloud data sent as notification value */
        eSetValueWithOverwrite );
      break;
      /* Other switch cases go here. */
    }
  }
}
```

Pseudo Write

```
/* Status bits used by the cloud write operation. */
#define SEND_SUCCESSFUL_BIT ( 0x01 << 0 )
#define OPERATION_TIMED_OUT_BIT ( 0x01 << 1 )
#define NO_INTERNET_CONNECTION_BIT ( 0x01 << 2 )
#define CANNOT_LOCATE_CLOUD_SERVER_BIT ( 0x01 << 3 )

/* A mask that has the four status bits set. */
#define CLOUD_WRITE_STATUS_BIT_MASK ( SEND_SUCCESSFUL_BIT |
                                      OPERATION_TIMED_OUT_BIT |
                                      NO_INTERNET_CONNECTION_BIT |
                                      CANNOT_LOCATE_CLOUD_SERVER_BIT )

uint32_t CloudWrite( uint32_t ulDataID, uint32_t ulDataValue )
{
  CloudCommand_t xRequest;
  uint32_t ulNotificationValue;

  /* Set the CloudCommand_t structure members to be correct for this write request. */
  xRequest.eOperation = eWrite; /* This is a request to write data */
  xRequest.ulDataID = ulDataID; /* A code that identifies the data being written */
  xRequest.ulDataValue = ulDataValue; /* Value of the data written to the cloud server. */
  xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle(); /* Handle of the calling task. */

  /* Clear the three status bits relevant to the write operation by calling xTaskNotifyWait()
```

```
  with the ulBitsToClearOnExit parameter set to CLOUD_WRITE_STATUS_BIT_MASK, and a block time
of 0. The current notification value is not required, so the pulNotificationValue parameter is
set to NULL. */
  xTaskNotifyWait( 0, CLOUD_WRITE_STATUS_BIT_MASK, NULL, 0 );

  /* Send the request to the server task. */
  xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

  /* Wait for a notification from the server task. The server task writes a bitwise status
  code into this task's notification value, which is written to ulNotificationValue. */
  xTaskNotifyWait( 0, /* No bits cleared on entry. */
    CLOUD_WRITE_STATUS_BIT_MASK, /* Clear relevant bits to 0 on exit. */
    &ulNotificationValue, /* Notified value. */
    pdMS_TO_TICKS( 250 ) ); /* Wait a maximum of 250ms. */

  /* Return the status code to the calling task. */
  return ( ulNotificationValue & CLOUD_WRITE_STATUS_BIT_MASK );
}
```

```
void ServerTask( void *pvParameters )
{
  CloudCommand_t xCommand;
  uint32_t ulBitwiseStatusCode;

  for( ;; )
  {
    /* Wait for the next message. */
    xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );
    /* Was it a read or write request? */
    switch( xCommand.eOperation )
    {
    case eWrite:
    /* Send the data to the remote cloud server. SetCloudData() returns a bitwise status code
       that only uses the bits defined by the CLOUD_WRITE_STATUS_BIT_MASK definition. */
      ulBitwiseStatusCode = SetCloudData( xCommand.ulDataID, xCommand.ulDataValue );

    /* Send a notification to the task that made the write request.
       The eSetBits action is used so any status bits set in ulBitwiseStatusCode will be set
       in the notification value of the task being notified.
       All the other bits remain unchanged.
       The handle of the task is obtained from the CloudCommand_t structure. */
      xTaskNotify( xCommand.xTaskToNotify, /* The task's handle is in the structure. */
        ulBitwiseStatusCode, /* Cloud data sent as notification value. */
        eSetBits );
      break;
      /* Other switch cases go here. */
    }
  }
}
```

10 Low Power Support

The FreeRTOS tickless idle mode stops the periodic tick interrupt during idle periods until either an interrupt occurs, or it is time for the RTOS kernel to transition a task into the Ready state.

sleep mode	Description
eAbortSleep	This mode denotes a task has been made ready, a context switch was pended or a tick interrupt has already occurred but was pended since the scheduler was suspended. It signals the RTOS to abort entering a sleep mode.
eStandardSleep	This mode allows to enter a sleep mode that will not last longer than the expected idle time.
eNoTasksWaitingTimeout	This mode is entered when no tasks are waiting for a timeout so it is safe to enter a sleep mode that can only be exited by an external interrupt or reset.

Function	Description
portSUPPRESS_TICKS_AND_SLEEP()	When the tickless idle functionality is enabled, the kernel will call it.
vPortSuppressTicksAndSleep()	it can be used to implement the tickless mode, can be overridden by the application writer.
eTaskConfirmSleepModeStatus()	It returns the sleep mode status, “eSleepModeStatus”.
configPRE_SLEEP_PROCESSING()	Before the user can make the MCU enter the low-power mode, it must be called to configure the system parameters to reduce the system power consumption
configPOST_SLEEP_PROCESSING()	After exiting the low-power mode, the user should call the this function to restore the system's main frequency and peripheral functions.

```
/* First define the portSUPPRESS_TICKS_AND_SLEEP() macro. The parameter is the time, in ticks,
until the kernel next needs to execute. */
#define portSUPPRESS_TICKS_AND_SLEEP( xIdleTime ) vApplicationSleep( xIdleTime )

/* Define the function that is called by portSUPPRESS_TICKS_AND_SLEEP(). */
void vApplicationSleep( TickType_t xExpectedIdleTime )
{
  unsigned long ulLowPowerTimeBeforeSleep, ulLowPowerTimeAfterSleep;
  eSleepModeStatus eSleepStatus;

  /* Read the current time from a time source that will remain operational while the
     microcontroller is in a low power state. */
  ulLowPowerTimeBeforeSleep = ulGetExternalTime();

  /* Stop the timer that is generating the tick interrupt. */
  prvStopTickInterruptTimer();

  /* Enter a critical section that will not effect interrupts bringing the MCU
     out of sleep mode. */
  disable_interrupts();

  /* Ensure it is still ok to enter the sleep mode. */
  eSleepStatus = eTaskConfirmSleepModeStatus();
  if( eSleepStatus == eAbortSleep )
  {
    /* A task has been moved out of the Blocked state since this macro was executed,
       or a context siwth is being held pending. Do not enter a sleep state.
       Restart the tick and exit the critical section. */
    prvStartTickInterruptTimer();
```

```
    enable_interrupts();
  }
  else
  {
    if( eSleepStatus == eNoTasksWaitingTimeout )
    {
      /* It is not necessary to configure an interrupt to bring the microcontroller
         out of its low power state at a fixed time in the future. */
      prvSleep();
    }
    else
    {
      /* Configure an interrupt to bring the microcontroller out of its low power state
        at the time the kernel next needs to execute. The interrupt must be generated from a
        source that remains operational when the microcontroller is in a low power state. */
      vSetWakeTimeInterrupt( xExpectedIdleTime );

      /* Enter the low power state. */
      prvSleep();

      /* Determine how long the microcontroller was actually in a low power state for,
      which will be less than xExpectedIdleTime if the microcontroller was brought out
      of low power mode by an interrupt other than that configured by the
      vSetWakeTimeInterrupt() call.
      Note that the scheduler is suspended before portSUPPRESS_TICKS_AND_SLEEP() is called,
      and resumed when portSUPPRESS_TICKS_AND_SLEEP() returns. Therefore no other tasks will
      execute until this function completes. */
      ulLowPowerTimeAfterSleep = ulGetExternalTime();

      /* Correct the kernels tick count to account for the time the microcontroller
         spent in its low power state. */
      vTaskStepTick( ulLowPowerTimeAfterSleep - ulLowPowerTimeBeforeSleep );
    }
    /* Exit the critical section
     - it might be possible to do this immediately after the prvSleep() calls. */
    enable_interrupts();
    /* Restart the timer that is generating the tick interrupt. */
    prvStartTickInterruptTimer();
  }
}
```

The Idle task can optionally call an application defined hook (or callback) function - the idle hook.

The idle task runs at the lowest priority,

This makes the Idle hook function an ideal place to put the processor into a low power state

- providing an automatic power saving whenever there is no processing to be performed.

```
void vApplicationIdleHoot( void );
```

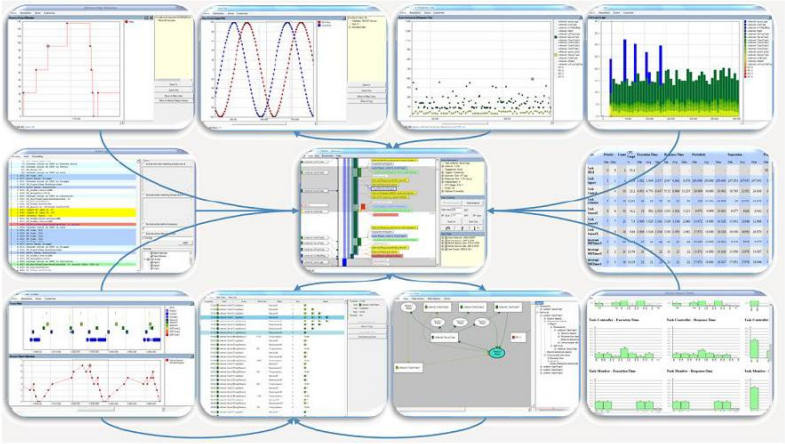
11 Developer Support

11.1 configASSERT()

C	FreeRTOS
assert()	configASSERT()

11.2 Tracealyzer

Tracealyzer is a run-time diagnostic and optimization tool provided by Percepio.



main trace view
CPU load view
response time view
user event plot view
kernel object history view

11.3 Debug Related Hook (Callback) Functions

Defining a **malloc failed hook** ensures the application developer is notified immediately if an attempt to create a task, queue, semaphore or event group fails.
Defining a **stack overflow hook** ensures the application developer is notified if the amount of stack used by a task exceeds the stack space allocated to the task.

11.4 Viewing Run-time and Task State Information

	Description	
Task run-time statistics	provide information on the amount of processing time each task has received.	
	To obtain binary run-time statistics information	uxTaskGetSystemState()

	To obtain run-time statistics information as a human readable ASCII table: vTaskGetRunTimeStatistics()
The Run-Time Statistic Clock	It is recommended to make the frequency of the run-time statistics clock between 10 and 100 times faster than the frequency of the tick interrupt.
Configuring an Application to Collect Run-Time Statistics	portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() portGET_RUN_TIME_COUNTER_VALUE(), or portALT_GET_RUN_TIME_COUNTER_VALUE(Time)
uxTaskGetSystemState() API Function	uxTaskGetSystemState() provides a snapshot of status information for each task under the control of the FreeRTOS scheduler.
vTaskListTasks() Helper Function	vTaskListTasks() provides similar task status information to that provided by uxTaskGetSystemState(), but it presents the information as a human readable ASCII table, rather than an array of binary values. tcpip R 3 393 0 Tmr Svc R 3 111 48 QConsB1 R 1 143 3 QProdB5 R 0 144 7 QConsB6 R 0 143 8 PolSEM1 R 0 145 11 PolSEM2 R 0 145 12 GenQ R 0 155 17 MuLow R 0 147 18 Rec3 R 0 141 30 SUSP_RX R 0 148 36 Math1 R 0 167 38 Math2 R 0 167 39
vTaskGetRunTimeStatistics() Helper Function	vTaskGetRunTimeStatistics() formats collected run-time statistics into a human readable ASCII table. PolSEM1 994 <1% PolSEM2 23248 1% GenQ 194479 16% MuLow 3690 <1% Rec3 229450 18% CNT1 242720 19% PeekL 94 <1% CNT_INC 165 <1% CNT2 243166 20% SUSP_RX 243192 20% IDLE 55 <1%
Generating and Displaying Run-Time Statistics, a Worked Example	Trace macros are macros that have been placed at key points within the FreeRTOS source code.

11.5 Trace Hook Macros

the most commonly used trace hook macros

function	Descriptions
traceTASK_INCREMENT_TICK(xTickCount)	Called during the tick interrupt, before the tick count is incremented. xTickCount passes the new tick count value
traceTASK_SWITCHED_OUT()	Called before a new task is selected to run. pxCurrentTCB contains the handle of the task about to leave the Running state.
traceTASK_SWITCHED_IN()	Called after a task is selected to run.

	pxCurrentTCB contains the handle of the task about to enter the Running state.
traceBLOCKING_ON_QUEUE_RECEIVE(pxQueue)	Called immediately before the currently executing task enters the Blocked state following an attempt to read from an empty queue, or an attempt to 'take' an empty semaphore or mutex. The pxQueue parameter passes the handle of the target queue or semaphore into the macro.
traceBLOCKING_ON_QUEUE_SEND(pxQueue) traceQUEUE_SEND(pxQueue) traceQUEUE_SEND_FAILED(pxQueue) traceQUEUE_RECEIVE(pxQueue) traceQUEUE_RECEIVE_FAILED(pxQueue) traceQUEUE_SEND_FROM_ISR(pxQueue) traceQUEUE_SEND_FROM_ISR_FAILED(pxQueue) traceQUEUE_RECEIVE_FROM_ISR(pxQueue) traceQUEUE_RECEIVE_FROM_ISR_FAILED(pxQueue) traceTASK_DELAY_UNTIL(xTimeToWake) traceTASK_DELAY()	…

Each trace macro has a default empty definition. The default definition can be overridden by providing a new macro definition.

Plug-ins that provide some FreeRTOS awareness are available for the following IDEs.

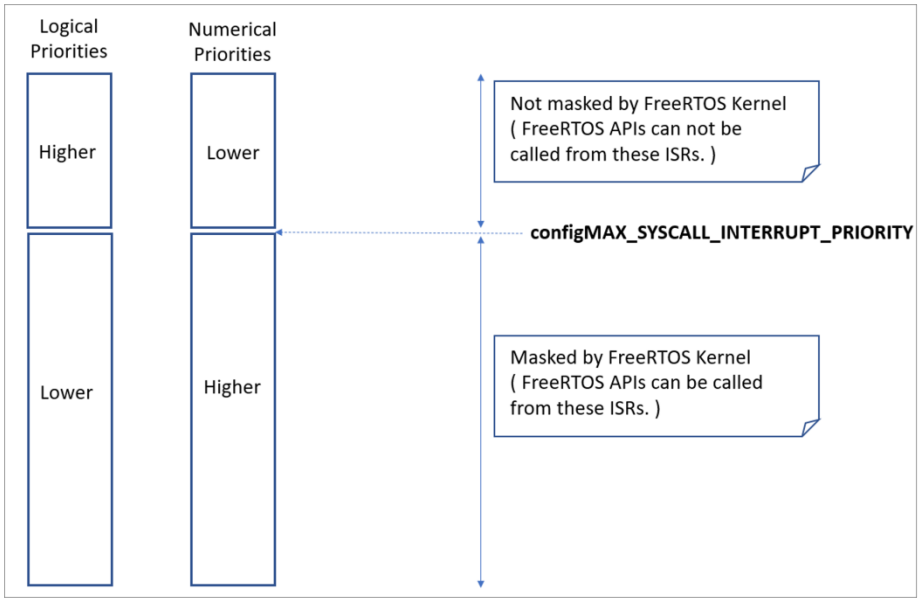
Eclipse (StateViewer) Eclipse (ThreadSpy) IAR ARM DS-5 Atollic TrueStudio Microchip MPLAB iSYSTEM WinIDEA STM32CubeIDE

12 Troubleshooting

Incorrect interrupt priority assignment, stack overflow, and inappropriate use of printf()

12.1 Interrupt Priorities

Logical and Numerical priority mean are reversed.



12.2 Stack Overflow

uxTaskGetStackHighWaterMark() is used to query how close a task has come to overflowing the stack space allocated to it. This value is called the stack 'high water mark'.

To use a stack overflow hook function:

void vApplicationStackOverflowHook()	The stack overflow hook (or stack overflow callback) is a function that is called by the kernel when it detects a stack overflow.

Run Time Stack Checking

Method 1	The kernel **checks that the stack pointer remains** within the valid stack space **after the context has been saved**. The stack overflow hook is called if the stack pointer is found to be outside its valid range. Method 1 is quick to execute, but can miss stack overflows that occur between context switches.
Method 2	When a task is created, its stack is filled with a known pattern. (DEADBEEF mark) Method 2 **tests the last valid 20 bytes of the task stack space** to verify that this pattern has not been overwritten. The stack overflow hook function is called if any of the 20 bytes have changed from their expected values.
Method 3	This method **enables ISR stack checking**. When an ISR stack overflow is detected, an assert is triggered.

12.3 Use of printf() and sprintf()

printf() and sprint() probably won't be suitable for use inside an interrupt service routine, **printf-stdarg.c** provides a minimal and stack-efficient implementation of sprintf()

12.4 Other Common Sources of Error

Symptom	Reason and solutions
Adding a simple task to a demo causes the demo to crash	Many of the demo application projects dimension the heap to be exactly big enough to create the demo tasks—so, after the tasks are created, there will be insufficient heap remaining for any further tasks, queues, event groups, or semaphores to be added.
Using an API function within an interrupt causes the application to crash	Use API that has "_FromISR()" prefix.
Sometimes the application crashes within an interrupt service routine	The first thing to check is that the interrupt is not causing a stack overflow. Some ports only check for stack overflow within tasks, and not within interrupts.
The scheduler crashes when attempting to start the first task	Ensure the FreeRTOS interrupt handlers have been installed. Some processors must be in a privileged mode before the scheduler can be started..
Interrupts are unexpectedly left disabled, or critical sections do not nest correctly	If a FreeRTOS API function is called before the scheduler has been started then interrupts will deliberately be left disabled, and not re-enabled again until the first task starts to execute.
The application crashes even before the scheduler is started	An interrupt service routine that could potentially cause a context switch must not be permitted to execute before the scheduler has been started. It is best to restrict API usage to the creation of objects such as tasks, queues, and semaphores, rather than the use of these objects, until after vTaskStartScheduler() has been called.
Calling API functions while the scheduler is suspended, or from inside a critical section, causes the application to crash	Do not call API functions while the scheduler is suspended, or from inside a critical section.

12.5 Additional Debugging Steps

If you encounter an issue not covered in the common causes described above, you can try to use some of the following debugging steps.

- Define configASSERT(), enable malloc failed checking and stack overflow checking in the application's FreeRTOSConfig file.
- Check the return values of the FreeRTOS APIs to make sure those were successful.
- Check that the secheduler related configuration, like configUSE_TIME_SLICING, and configUSE_PREEMPTION are set correctly as per the application requirements.